

Machine Learning Literacy for Measurement Professionals: A Practical Tutorial

Rui Nie , Qi Guo , and Maxim Morin , Medical Council of Canada

Abstract: The COVID-19 pandemic has accelerated the digitalization of assessment, creating new challenges for measurement professionals, including big data management, test security, and analyzing new validity evidence. In response to these challenges, Machine Learning (ML) emerges as an increasingly important skill in the toolbox of measurement professionals in this new era. However, most ML tutorials are technical and conceptual-focused. Therefore, this tutorial aims to provide a practical introduction to ML in the context of educational measurement. We also supplement our tutorial with several examples of supervised and unsupervised ML techniques applied to marking a short-answer question. Python codes are available on GitHub. In the end, common misconceptions about ML are discussed.

Keywords: automated marking, data science, educational measurement, machine learning, tutorial

In the 21st century, educational assessments have become increasingly digitalized; a phenomenon accelerated more recently due to the COVID-19 pandemic. Many assessment organizations were forced to move their examinations to remote online settings. This massive paradigm shift created many challenges and opportunities. For example, validity evidence needs to be re-evaluated using new forms of data (e.g., big process data, videos), test security concerns have become increasingly pronounced due to remote testing, and previous human resources may not suffice to accommodate the greater demands in examination content and operations.

With a more significant workload and (potentially) fewer resources, *Machine Learning* (ML) has become an increasingly appealing tool for dealing with challenges in modern educational measurement. Even before the pandemic, the ML revolution has already impacted most, if not all, disciplines. For example, in educational measurement, researchers and practitioners have successfully applied ML in various areas, including content validity (Anderson et al., 2020), item/test development (Rafatbakhsh et al., 2021), test security (Ferrara, 2017), marking/scoring (Ercikan & McCaffrey, 2022), and crisis prediction (Burkhardt et al., 2020), to name a few. Meanwhile, various new ML-based educational disciplines have emerged (e.g., educational data mining, learning analytics, and computational psychometrics; von Davier et al., 2021). Therefore, the authors believe *ML Literacy* (i.e., the ability to understand and apply the practical aspects of ML) is essential for educational measurement specialists to succeed in this new era.

However, most ML courses are designed for computer scientists. As a result, they tend to focus on mathematical derivations and algorithm implementations. However, in practice, most ML studies are conducted using well-established ML packages such as scikit-learn (Pedregosa et al., 2011). Technical implementations of ML algorithms are rarely needed. Accordingly, the authors believe measurement specialists can

first focus on understanding the purposes and applications of ML concepts rather than technical details. Using an analogy, most people do not need to know the mechanics of cars to drive, and those who know the mechanics may not be the best drivers. Therefore, this tutorial aims to introduce ML from a practical perspective: explaining ML using traditional statistical concepts that measurement specialists are familiar with, highlighting the workflow of a ML study without going into technical details, and demonstrating ML using applied examples in educational measurement.

The tutorial is divided into three sections. The first section introduces the basic ML concepts, processes and recommended programming language and software. The second section provides demonstrations of supervised and unsupervised ML using an automated marking example. The Python scripts used for this demonstration are provided on GitHub: <https://github.com/rui-nie/automark>. Finally, the third section discusses common misconceptions about ML and concludes the tutorial. The definitions for all the ML terminologies covered in this paper can be found in Appendix A.

Introduction to Machine Learning

Basic Concepts

A popular definition of ML is “a field of study that gives computers the ability to learn without being explicitly programmed” (Cui, 2021). Since this definition is relatively broad, it is helpful to compare it with familiar concepts in statistics to gain a better understanding. Traditional statistics consists of three major components: model specification, parameter estimation, and parameter probability distribution derivation. For example, in linear regression, researchers need to specify what variables to include in the model, compute regression parameters using an estimation method such as least squares, and test the significance of each

regression parameter based on its distribution. The derivation of parameter distribution allows traditional statistics to calculate parameter significance. However, it is also the most mathematics-demanding step that limits the complexity of the model specified in the first step.

In contrast, while ML also has the model specification and parameter estimation (i.e., Training) steps, most ML studies do not involve deriving parameter probability distribution. By removing the most mathematics-demanding step, ML is free to specify more complex models for better prediction power at the cost of ignoring parameter significance. Another important distinction is that model specification is theory-driven in traditional statistics, while the model specification is often data-driven in ML. As a result, ML researchers need to experiment with different model architectures to identify the best model for a problem. At last, many ML parameter estimation techniques are designed for big and sparse data. One way to characterize *big data* is by the 3 Vs (von Davier et al., 2021): large *volume* (i.e., large data size), wide *variety* (i.e., diverse data types: videos, graphics, texts, numerical, etc.), and high *velocity* (i.e., data coming in fast). Consequently, ML parameter estimation techniques often process data in many batches and are robust to variables with zero variances or perfect correlations.

Several similar concepts are related to ML, including *Artificial Intelligence* (AI), *data science*, *data mining*, and *Natural Language Processing* (NLP). AI is a broader term that includes all forms of intelligence demonstrated by machines (Shane & Marcus, 2007). For example, an *expert system*, a computer system emulating human problem-solving behaviors using preprogrammed if-then rules, is a form of AI that is not ML. Data mining and data science use ML and other statistical and scientific techniques to extract knowledge from data (Hao & Mislevy, 2021). At last, NLP use AI, statistics, psychology, and linguistics to enable computers to process and generate human language automatically (Flor & Hao, 2021). All these concepts are relevant for this tutorial, but the focus will be on ML.

There are two major types of ML¹: *supervised* and *unsupervised* ML. Supervised ML focuses on predicting the *label* variable using the *feature* variables. In statistical terms, the label variable is the dependent variable, and the feature variables are the independent variables. Additionally, an entry/row of data is called a *record*. Predicting a categorical label variable is called *classification*, while predicting a continuous label variable is called *regression*. An example of a basic regression model is linear regression, and an example of a basic classification model is logistic regression. More advanced supervised models, such as artificial neural networks and support vector machines, extend basic supervised models by extracting nonlinear features from the input feature variables and using these nonlinear features to predict the label.

Three popular classes of supervised learning algorithms are bagging algorithms (e.g., random forest; Breiman, 2001), boosting algorithms (e.g., *CatBoost*, *XGBoost*, & *LightGBM*; Hao, 2021), and deep neural networks (also called deep learning, e.g., *convolutional neural networks*, *recurrent neural networks*, *transformers*; Huang & Khan, 2021). In general, bagging and boosting algorithms are efficient for structured tabular data types (i.e., continuous and categorical variables), while deep learning models are better suited for special data types such as images, videos, and big textual data.

While supervised learning algorithms have the capacity to model highly nonlinear relationships, they may also cap-

ture irrelevant noises in the sample data. This phenomenon is called *overfitting*. To prevent overfitting, researchers can constrain the model parameters to be as small as possible while maximizing accuracy. This forces nonessential parameters to be close to 0, reducing noises in the model. This method is called *regularization* (Hao, 2021).

Unsupervised ML focuses on modeling data without labels by creating latent variables (e.g., clusters, factors, components; Wong, 2021). In statistics, latent variables are unobserved variables inferred from other observed variables. In the context of ML, they help group features or records. Basic techniques include principal component analysis for grouping features and cluster analysis for grouping records. Again, these techniques are commonly used in educational measurement research. While there are numerous unsupervised algorithms in ML, most can be classified as either dimension reduction or cluster analysis techniques.

Machine Learning Workflow

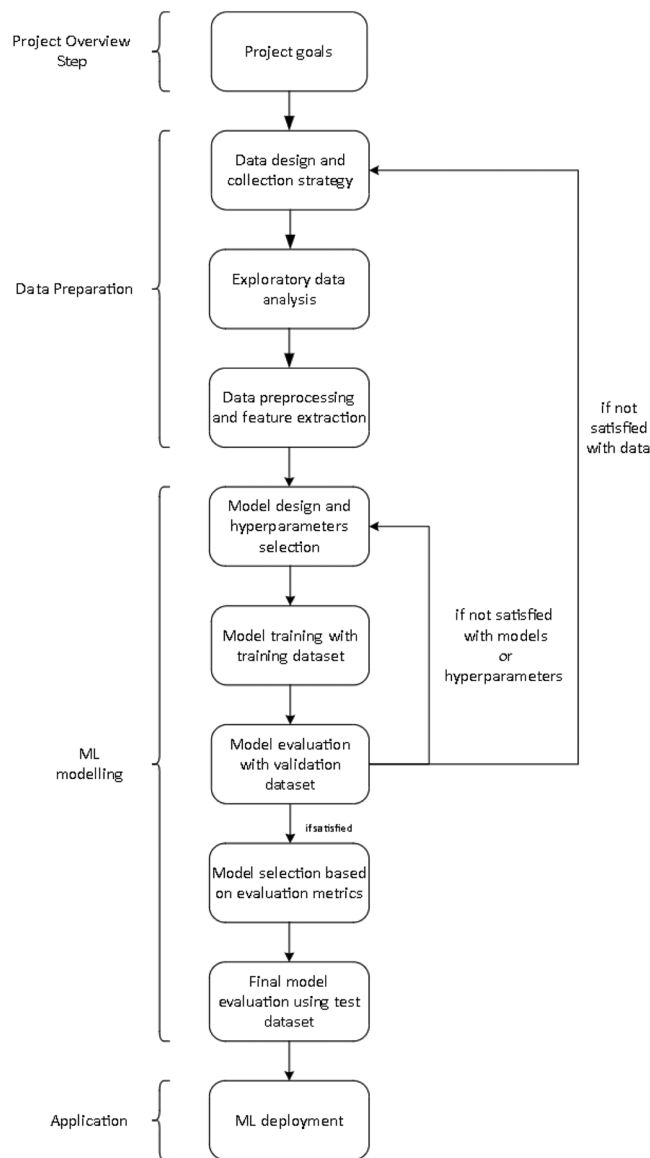
A ML study consists of several major steps, forming a workflow. We present a general ML workflow in Figure 1 and describe each component in the following sections.

Project goals and data design. A ML study begins with clear definitions of project goals and a proper data design and data collection strategy. A ML study will fall short without good-quality data. Therefore, careful planning of the data model and the data collection strategy can prevent many issues in later data analysis (Hao & Mislevy, 2021). For example, to develop a ML solution for automated scoring, it may be helpful to follow a framework such as the *Evidence Centered Design* (ECD) framework (Mislevy et al., 2003; Ercikan & McCaffrey, 2022). Under the ECD framework, measurement specialists need to (1) specify detailed responses/behaviors associated with each component of the targeted measurement construct (i.e., specifying the *construct model*), (2) ensure measurement tasks explicitly require examinees to perform construct-relevant behaviors (i.e., specifying the *task model*), and (3) use construct-relevant features in ML models to predict the final essay score (i.e., specifying the *evidence model*).

Exploratory data analysis. An important ML study task is to explore the data set to better understand the data. This process is called *Exploratory Data Analysis* (EDA). Like traditional statistical analysis, ML EDA often uses descriptive statistics and graphs to obtain information such as sample size, variable count, types, distributions, missing data, outliers, and correlations.

Data preprocessing and feature extraction. Raw data often need to be transformed before being used in a ML model. This step is called *Data Preprocessing*. For example, it is common to convert nominal variables (e.g., category labels) into numerical variables during data preprocessing. One hot coding, which is analogous to dummy coding in statistics, creates an indicator variable for each value in a nominal variable (e.g., 1 for the targeted value, 0 otherwise). In contrast, a more sophisticated algorithm, such as the categorical variable coding algorithm used in *CatBoost* can combine multiple nominal variables into one numerical variable and encodes label distribution information into the numerical variable (Prokhorenkova et al., 2017). When some feature variables have large variances, it is common to standardize the

Figure 1
Machine Learning Workflow



variables or scale the variables to have a range between 0 to 1. When missing data are present, *imputation* is often used to replace missing values with a predicted value. Sometimes, additional variables are created to code which values are missing (e.g., Savi et al., 2021). When there are too many features, *dimension reduction* (e.g., Principal Component Analysis or Latent Semantic Analysis) or *feature selection* (e.g., stepwise regression) techniques are often used to reduce the number of feature variables.

Text data are naturally unstructured and require special preprocessing. First, punctuations and commonly used words (i.e., *stop words*; e.g., the, I, a, is, etc.) can be removed. Second, different forms of a word can be converted to their base form (i.e. *stemming & lemmatization*; e.g., is, are, am → be; Flor & Hao, 2021). Third, pronouns and other similar expressions can be converted to the primary subject entity they are referring to in the text (i.e., *coreference resolution*; Clark & Manning, 2016; e.g., “Tom went to the game. He said it was

amazing.” → “Tom went to the game. [Tom] said [the game] was amazing.”).

Specific feature extraction techniques are also applied to text data. That can be done by converting text data into a feature vector of word frequencies (i.e., the *Bag of Words Model*; Flor & Hao, 2021). More generally, the frequency of N consecutive word phrases (the word N -gram model) can be computed. For example, the sentence “The car is blue” can be divided into 4 unigrams (“The,” “car,” “is,” “blue”) and three bi-grams (“The car,” “car is,” “is blue”). Character n -grams can also be counted. For example, character bi-grams of the word “car” are “ca” and “ar.” Besides word frequency, other statistics can be used. For example, *Term Frequency-Inverse Document Frequency* (TFIDF; Flor & Hao, 2021) is a popular statistic representing the importance of a word in a collection of texts. An alternative encoding approach is *text embedding*, which converts texts into a high-dimensional numeric vector representing the meaning of the text (Pennington et al., 2014). Many pretrained text embeddings are available; they are usually obtained by training NLP models on large text corpora (e.g., Wikipedia, Twitter). Pretrained NLP models can also be applied to the texts to create theoretically meaningful features. For example, in automated essay scoring, theoretical meaningful features such as the number of grammatical errors (Almusharraf & Alotaibi, 2020), text complexity (Sheehan, 2017), and text cohesion metrics (McNamara et al., 2010) can be first extracted using various pretrained/theoretically defined NLP models. The features can later be used in more specific ML models to score essays. This approach of applying pretrained models/parameters to different but related problems is called *transfer learning*. While the NLP models and techniques mentioned in this section can be daunting for beginners, in practice, they can be directly applied to text data without knowing the technical details.

Model selection and evaluation. The previous section introduced various data preprocessing steps to transform raw data into a list of feature variables. These are then input into a ML model to predict the label (supervised learning) or create latent features (unsupervised learning). Since there are many good-performing ML models, it is common to train and compare several models to identify the best one for the problem. Therefore, it is important to establish a strategy to specify ML models, evaluate them, and select the best candidate.

The formal strategy starts with building a ML *pipeline*, which consists of a sequence of steps: data preprocessing, feature extraction, model training, and evaluation. Many parameters can be adjusted inside a pipeline to produce the desired result. In ML, we often distinguish between the *hyperparameters* specified before training the model and the *parameters* learned during the training process. To draw a parallel with statistical modeling, in a K -means clustering model, researchers need to specify the number of clusters, k , the hyperparameter, before training the model. After training the models, cluster centroids are learned as the model parameters.

ML researchers often use a data-driven approach, such as *Grid Search* (san Pedro & Baker, 2021) and *Random Search* (Bergstra & Bengio, 2012), to find the best pipeline hyperparameters. In a grid search, the researcher would specify a list of testing values of each hyperparameter in the pipeline, then all combinations of the hyperparameters are tested. For example, in a text classification problem, a ML researcher may

be interested in testing two sets of stop words, comparing unigrams and bigrams, and adjusting the model complexity with two sets of model hyperparameters. In a random search, each hyperparameter in a pipeline is randomly generated based on the prior distribution specified by the researcher, and this process is repeated N times. Grid search can be used when a pipeline's hyperparameters have small numbers of discrete values, and computational time is not a significant concern. On the other hand, random search is more efficient when a pipeline's hyperparameters have large numbers of values, and the ML model is slow to train (e.g., deep learning).

Different ML pipelines are then evaluated and compared using various metrics. The most intuitive evaluation metric for classification problems is *accuracy*, the percentage of times a model correctly predicts the label. However, there are several situations where accuracy may be a poor evaluation metric. First, accuracy can be spuriously high when the distribution of the label variable is very unbalanced. For example, when trying to detect a rare disease, the accuracy would remain high even if a model classifies every record as false. Many other evaluation metrics have been proposed and are commonly used to correct this problem. Statistics such as *Kappa* for binary labels and *Quadratic Kappa* for ordinal labels (Cohen, 1960, 1968) were derived to consider the label variable's distribution; while other statistical methods and measures, such as a confusion matrix, *F-score* and the *Receiver Operating Characteristic (ROC) curve* are commonly used when considering error types. That is particularly helpful when researchers want to control a specific error type (e.g., detecting all COVID cases at the cost of misdiagnosing healthy people or minimizing misdiagnosing healthy people at the cost of failure to detect some COVID cases). For example, the *ROC curve* has been developed to evaluate a model at different classification thresholds (Fawcett, 2006). And the *Area Under the ROC Curve (AUC)* is used to evaluate a model's overall performance across thresholds.

At last, overfitting the training data is a critical consideration when evaluating ML models (Hao, 2021). To select the most generalizable model, ML researchers often randomly divide a data set into training, validation, and test sets. First, the training set is used to train the ML models/pipelines and fit model parameters. Next, the trained models/pipelines are evaluated using the validation set. Based on validation set performance, the best model/pipeline (with the best set of hyperparameters) is selected, and its performance on the test set is reported as the final evaluation of the model. When the training-validation split occurs only once, and only one validation set is created, it is called a *hold-out validation*. That strategy is often used when computation is time-consuming. However, how to split a data set into training and validation sets can be arbitrary, so when computation is relatively fast, *k-fold cross-validation* (Hao, 2021) is usually the preferred method. It is a popular validation method that circumvents the problem by randomly dividing a data set into k equal subsets and training the model in multiple cycles. In each cycle, a different subset is used as the validation set, and the other $k - 1$ subsets are used for training the model. In the end, the model's performance is averaged over all cycles.

After the best ML model is identified, *feature importance* scores can be calculated to help interpret the model. A feature's importance score represents the contribution of that feature to model prediction. Some feature importance statistics depend on the ML algorithm, and others are generic. For example, *GINI importance* is specifically designed for the de-

cision tree algorithm, while *permutation importance* can be computed for various ML models (Breiman, 2001).

Programming Language and Tools

Python is one of the most popular ML and general programming languages, with numerous packages and large and active support communities. Since Python is, first and foremost, a general programming language, it is very suitable to implement ML solutions for industries. R programming language also offers several ML packages. Compared with Python, R was initially designed for statistical programming and does not necessarily offer all the flexibility and functionalities of a general programming language. However, it has many statistics and psychometrics packages, which makes it popular among educational measurement professionals and an excellent tool for academic research and education.

Programmers often use *Integrated Development Environments (IDEs)* to code more efficiently. IDEs have various utilities for installing packages, organizing codes, refactoring codes (efficiently modifying codes), debugging (i.e., identifying and resolving errors), testing (i.e., running a series of tests to ensure the codes meet expectations), and version control (i.e., tracking and managing different versions of the codes). Choosing a good IDE is essential for professional project development. We recommend using PyCharm or Visual Studio Code for Python projects and Rstudio for R projects. Note that different IDEs are developed for diverse purposes. For example, Jupyter Notebook is a great Python IDE for educational/data analysis purposes, while PyCharm is an excellent tool for formal project development.

For educational researchers who prefer a Graphical User Interface (GUI) over coding, WEKA is a great and free tool for conducting ML analysis (Frank et al., 2016). Compared with Python, WEKA does not require users to have a programming background; therefore, it is great for teaching and conducting simple ML analysis. In contrast, Python is more flexible and efficient for complex projects.

ML Communities and Resources

There are various online ML communities and resources. For overall guidance, GitHub offers a roadmap² for beginners to navigate various online ML resources and communities. For ML courses, Udemy offers practical courses, Coursera offers academic courses, and YouTube offers various free courses. For ML tutorials, Towards Data Science and Medium³ are great websites. For ML-related questions, there are many online communities to help, such as Reddit⁴ and Stack Overflow.⁵ For ML data, Kaggle offers data sets from diverse disciplines and allows ML researchers to compete for the best-performing model. For sharing NLP models, Hugging Face (Wolf et al., 2020) is a great resource for state-of-the-art NLP models.

Experimental Study

This section demonstrates the application of some popular supervised and unsupervised ML methods in developing automated marking solutions for short-answer questions.

Problem Description

The question selected for these examples comes from a national medical licensing exam. In this exam, the examinees are presented with a clinical scenario with one or more questions and are asked to provide one or multiple short written responses to each question. Two human markers then evaluate each response's accuracy by matching it to the unique answer key. If a response does not match any given correct answer's answer key, the marker will match it to a dedicated "incorrect" answer key. If the two markers disagree, a super marker will resolve the discrepancy and make a final decision. The question scores are calculated automatically based on predetermined rules. The marking process described is labor-intensive and vulnerable to several threats (fatigue, individual bias, drift over time, etc.). Thus, using automated marking to assist with the marking (e.g., replacing one human marker) is an appealing solution to reduce costs and improve reliability. The examples provided in this tutorial were part of a larger project that aimed to develop ML supervised and unsupervised solutions to classify short answers. For the sake of demonstration, we applied the ML solutions to a single short-answer question, and we deliberately selected a question with a relatively low ML accuracy.

The short-answer question was administered on a national medical exam that assesses a candidate's critical medical knowledge and clinical decision-making ability at a level expected of a medical student who is completing their medical degree (construct model). For each clinical case and question, subject-matter experts specify a list of appropriate clinical decisions, which are reflected in the list of acceptable and unacceptable answers (i.e., answer keys). The clinical problem, leading question and answer keys selected for this demonstration are displayed below.

A 23-year-old woman, gravida 2, para 1, aborta 0, comes to the office for a 22-week prenatal visit. She is tearful and says that she cannot go through with the pregnancy. For several days, she has felt that she is not a very good mother to her 3-year-old daughter. She and her husband argue. She states that he is not supportive and that she "cannot please him anymore." The patient has difficulty sleeping, and her appetite has been poor. She also has difficulty concentrating and feels tired most of the time. She has no thoughts of harming herself or her family. She is physically healthy and moderately active. She had similar symptoms for 2 months after the birth of her first child. How will you manage this patient's care? List up to 2, or type in "None" if no management is indicated.

Answer key 1: Psychotherapy. **Synonyms:** Cognitive behavior therapy; interpersonal therapy; referral to a psychologist; referral to a psychiatrist; referral to mental health services; supportive therapy; group therapy. **Not Acceptable:** Biofeedback; counseling.

Answer key 2: Family therapy. **Synonyms:** Marital therapy; couple's counseling; marital counseling; couple therapy; referral to a social worker; supportive therapy. **Not Acceptable:** Biofeedback.

Answer key 3: Incorrect answer.

Data Set

The demo data set is composed of one feature variable and one label variable. The feature variable is text and contains all the responses provided by examinees to the selected question. The label variable is a numerical identifier (an-

swerkey_id) that specifies the answer key assigned by the super marker to each text response. The data set includes 750 records. The text responses to the selected question are relatively short, with a median number of words of 4 (first quartile: 2, third quartile: 6). The "answerkey_id" variable takes three values, representing the first correct answer, the second correct answer, and incorrect answers, with a corresponding distribution of 25%, 7%, and 68%. The data set was randomly divided into training/test sets in a 7/3 ratio, stratified using the "answerkey_id" label.

In this tutorial paper, we conducted all analyses using real data. However, on GitHub, we simulated some fake data for confidentiality reasons.

Data Preprocessing

For data preprocessing, we used Python functions to automatically (1) convert letters into lowercase; (2) remove all punctuations, leading, trailing and extra white spaces; (3) lemmatize words with the spaCy lemmatizer (Honnibal & Montani, 2017).⁶

Machine Learning Pipelines

In this tutorial, we provide a Python script for each ML pipeline. The following sections are organized using the Python script names (e.g., supervised_simple.py) and describe each pipeline. For all non-deep learning pipelines, we used the scikit-learn software library (Pedregosa et al., 2011); for all deep learning pipelines, we used Keras API (Chollet et al., 2015) with TensorFlow (Abadi et al., 2016) as the backend.

Supervised learning: non-neural network pipelines. With large-scale assessment programs, it is very common to field test (or pilot test) items before using them on operational exams. That is particularly helpful with short-answer questions because it allows content developers and psychometricians to review and update answer keys. For ML applications, it also provides the opportunity to collect labeled data (marks or scores) and, therefore, to model the data using supervised learning techniques by predicting the label variable using the extracted features from the raw candidate responses. Below, we provide two examples of supervised learning pipelines.

Supervised_simple.py. This first example is a gentle introduction to building a machine learning pipeline using Python; it is also a common first step in the modeling phase: building a simple pipeline to establish the baseline. In this pipeline, word unigram is used as the input feature, and a random forest classifier with its default hyperparameter settings is used as the main model. They are easy to implement and interpret and usually work well in the text classification field, thus a good common choice for a first pipeline.

Supervised_search.py. After establishing a baseline, researchers usually start experimenting with different models and hyperparameter combinations to find the best-performing pipeline. The second script illustrates the use of a search pipeline, with both the grid search and the random search strategies, to demonstrate the automated model and hyperparameter search process. The search space consists of different choices of text preprocessing method in feature extraction step: word unigrams with or without excluding stop words, character n -grams with varying values of n (2–5); different counting and normalization method choices in feature

extraction step: basic bag-of-words counting method, with or without TFIDF weighting, and different choices of classifiers: CatBoost, linear Support Vector Machine (SVM), logistic regression, and random forest.

Supervised learning: deep neural network pipelines. Deep neural networks have a special position in ML, as they are currently the best-performing models in many challenging areas, such as machine translation and image recognition. In fact, in the text classification field, many current state-of-the-art models are deep learning models. One example is XLNet, which we will demonstrate in the third example. The sophistication of deep learning models gives them the capacity to process the original word/character sequence without transforming them into n -gram frequency features.

Because of the success of deep learning in text classification, we dedicate three examples to it in this section. The first two examples demonstrate how to construct a basic deep learning model, and the third example demonstrates how to apply a state-of-the-art model to our problem. The first two examples use the same deep learning model, the difference is only in input features: the first used word unigram, and the second used character n -grams. The reason is that we would like to compare which input features are best suited for the problem. The first and third examples also show two different use cases of transfer learning, which will be explained in detail later.

Supervised_search_cnn_word.py In this first example, we constructed a random search pipeline with word unigram as the input feature and a deep neural network as the model. More precisely, the main model is a *Convolutional Neural Network* (CNN) model, and the model's architecture is designed by the authors. This example demonstrates how to build a customized deep learning model. An in-depth introduction to CNN is out of the scope of this tutorial. For now, it is sufficient to understand that CNN is a class of deep neural networks inspired by the human visual system and is a common choice in solving text classification problems due to its good performance in this area.

In this example, we also demonstrate one use of transfer learning by using GloVe word embeddings (Pennington et al., 2014) as the initialization values of our own word embedding step. To be more specific, the GloVe word embeddings we used are pretrained on large text corpora from Wikipedia and Newswire. Using pretrained word embeddings allows us to apply the "knowledge" learned from large text corpora to our problem. As a result, the pipeline can have a better training start than just using random values.

The search space of this pipeline consists of different choices of hyperparameters related to network architecture, regularization and optimization.

Supervised_search_cnn_char.py. This example is the same as `Supervised_search_cnn_word.py`, except character n -grams were used instead of word unigrams as input. Character n -grams are robust to misspellings, but no pretrained embedding is available.

Supervised_xlnet.py. Neural network architectures can have endless varieties. Therefore, beginners are usually at a loss for what architecture to use. This is when state-of-the-art models can come to aid. Since their superior performance is already demonstrated with many data sets, there is a good chance it will perform well for the data set of interest. Whenever a new state-of-the-art model is published, it will usually

be implemented by major machine learning platforms very quickly. End users can directly import these models from machine learning libraries and use them without knowledge of their technical details.

These imported models come with parameters already pretrained on large data. During application, the researcher can have these parameters fixed or freed. If fixed, the model will be applied to data without any training; if freed, the pretrained parameters will only serve as initialization values, and the model will be trained on the researcher's data to better suit the problem at hand. Either way, this is another example of transfer learning.

In this example, we imported the pretrained XLNet model (Yang et al., 2020) from a Hugging Face (Wolf et al., 2020) library and applied it to our data. XLNet is a deep learning model and is one of the current state-of-the-art models in the text classification field. During application, we chose to free the model parameters to make the trained model better suit our data.

Unsupervised learning pipelines. While supervised learning algorithms can be applied to previously marked responses, they cannot be used with newly developed questions administered for the first time. That is because supervised learning requires labeled data to train model parameters. In this situation, unsupervised learning algorithms can be applied to augment human markers. For example, one promising solution for augmenting short-answer marking is to arrange candidate responses in small clusters. Clustering techniques, such as K -means clustering used in the following examples, aim to group data points with very similar features and differentiate between very dissimilar data points. With short-answer questions, we can hypothesize a reasonable number of clusters of responses in the space of all possible responses, and each cluster should have a distinct enough meaning. Then, the work of subject-matter experts can be directed toward these clusters and their meaning instead of using all candidate responses. For example, they can review clusters to determine the most common unique responses, compare them with the original answer key and improve it if needed. They can also mark the most representative response (i.e., centroid) for each cluster instead of every individual response. In this last Python script, we explore using K -means clustering as an unsupervised learning technique and evaluate its performance using the labeled data. The steps are as follows: (1) Group all examinees' responses into a fixed number of clusters using K -means clustering. (2) In each cluster, identify the response closest to the cluster centroid, and present only that response to human markers. (3) The answer key assigned to the representative response by the human marker is automatically given to the remaining responses in that cluster. (4) In the evaluation step, we evaluate the prediction of all the responses except the representative response in each cluster.

In practice, measurement professionals would use previously marked items to determine appropriate feature extraction methods (e.g., character or word n -grams) and hyperparameters (e.g., number of clusters, distance metrics) for the same type of exam questions. Then, the same settings can be applied to all new items (e.g., field test items). During the marking session, one marker could mark all the answers, while the second marker only marks the representative

Table 1*The Evaluation Results of Different Pipelines and Two “Non-Super” Human Markers on the Test Set*

Pipeline	Accuracy	Kappa	F-Score	ROC AUC
Supervised_simple.py	0.960	0.911	0.933	0.991
Supervised_search.py*	0.982	0.961	0.964	0.994
Supervised_search_cnn_word.py	0.920	0.817	0.818	0.978
Supervised_search_cnn_char.py	0.960	0.913	0.898	0.991
Supervised_xlnet.py	0.951	0.913	0.898	0.991
Unsupervised_search.py**	0.869	0.726	0.688	NA
Human marker 1	0.951	0.898	0.892	NA
Human marker 2	0.982	0.961	0.953	NA

*Supervised_search.py (in bold) exhibited the best performing evaluation metrics. **Based on the evaluation scheme in the “unsupervised learning pipelines” section, some labels in the testing set were considered as given by human markers and were excluded from the evaluation. Consequently, the unsupervised_search pipeline has a smaller test set size than other pipelines.

answers. Finally, any discrepancy is resolved by the super marker.

Unsupervised_search.py. In this script, we created a custom K -means clustering pipeline with a grid search. The search space consists of different choices of text preprocessing methods in the feature extraction step (word unigrams, character n -grams with n ranges from 2 to 4), two choices of counting and normalization method (basic bag-of-words counting method with or without TFIDF weighting), and two choices of the number of clusters (i.e., 20, 50). While more clusters tend to yield better accuracy, it also adds to the workload of human markers. So, when choosing the number of clusters, the objective is to minimize human workload while satisfying an acceptable accuracy threshold.

Evaluation Metrics and Feature Importance

We used accuracy as the single evaluation metric for selecting models in the above ML pipelines. However, during testing, we added other evaluation metrics, including ROC curve and AUC, F -score, Kappa, and confusion matrix, to further evaluate and understand the performance of our selected model. For the supervised non-neural network models, we also computed feature importance scores for all features to understand which are the most influential in the models.

For all non-deep learning pipelines presented in this tutorial, we used fivefold cross-validation, given the short training time. However, we used hold-out validation for deep learning pipelines as the training phase was much slower. More specifically, the training data were randomly split into training/validation sets in a 7/3 ratio, stratified by the “answerkey_id” label.

Demonstration Results and Discussion

Table 1 presents the evaluation metrics of different models (and the two human markers) on the test set. The ROC curves are shown in Figure 2 in separate panels. The best-performing pipeline on all four evaluation metrics and ROC curve (the curve is higher than other curves) is the CatBoost model with both bi and tri-character grams without TFIDF (i.e., the best pipeline from random search in supervised_search.py). That is not surprising because CatBoost is a powerful yet beginner-friendly algorithm, which does not require users to do extensive hyperparameter tuning. While with more extensive searches, it is possible for other algorithms to achieve better performance, the current result supports the

Table 2*Top 10 Features Based on Feature Importance Score for the Model in supervised_simple.py*

Top 10 Features	Feature Importance Score
Psychotherapy	0.125
Cognitive	0.088
CBT	0.073
Therapy	0.053
Couple	0.051
Behavioral	0.030
Psychiatry	0.028
Behavioral	0.027
Marriage	0.026
Referral	0.025

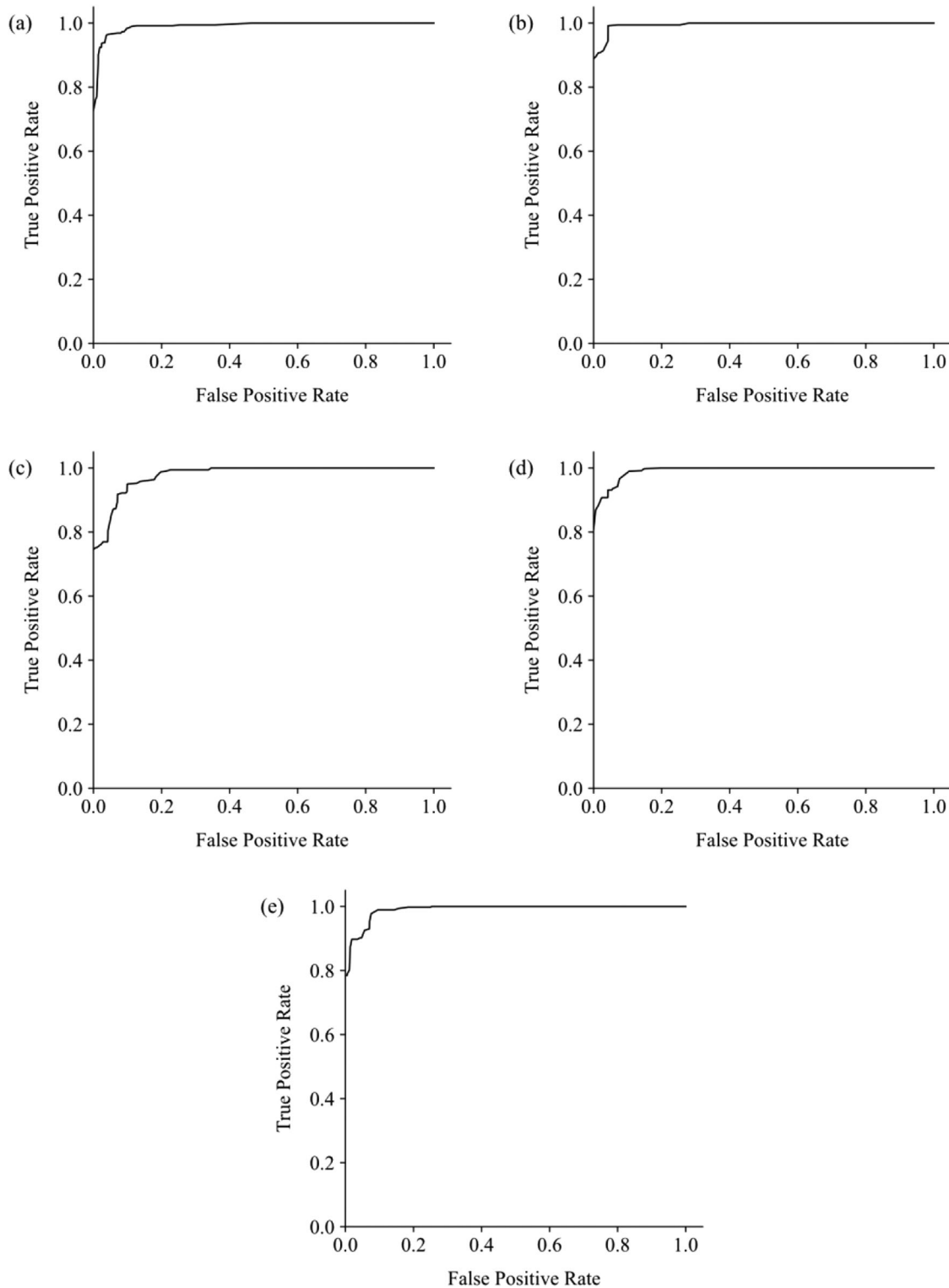
efficiency of the CatBoost algorithm for this problem. One trend in our experiment is that deep learning models tended to perform slightly worse than other models and were also slower to train. That may be because deep learning models are designed for complex problems and are not as efficient for more straightforward tasks. Another trend is that character-based models tended to outperform word-based models. That is likely because character-based models are robust to misspellings (human markers are told not to consider them). At last, we want to point out that most supervised methods have comparable or even better performance than human markers. One appealing property of supervised marking solutions is that they are consistent over time. This consistency is often harder to achieve for manual marking, for example, when different groups of markers are hired at each examination session.

The unsupervised methods performed worse than supervised learning on all metrics.⁷ That is expected because the unsupervised automated marking techniques are designed for new questions without previous marking data. Additionally, the question used for this demonstration has many different responses, making the unsupervised marking technique less effective. Unsupervised methods would probably perform better if a question had fewer unique responses.

To demonstrate the use of feature importance scores for model validation, we present the ten most important features for the model in supervised_simple.py in Table 2.

Figure 2

ROC Curve for Different Pipelines on the Test Set. (a) *supervised_simple.py* (b) *supervised_search.py* (c) *supervised_cnn_word.py* (d) *supervised_cnn_char.py* (e) *supervised_xlnet.py*



A word-based model is used for demonstration because character-based models' features are more difficult to interpret. As can be seen, the top features are all related to different forms of psychotherapies or couple therapies, which are consistent with the answer keys.

Discussion

This paper aimed to expand the toolkit of measurement specialists by providing a step-by-step tutorial on applying ML techniques. We focused on ML applications to the marking of short-answer questions. However, there are many other

potential applications of ML for assessment and measurement problems.

In this tutorial, we show that there are many parallels between ML and assessment science. In addition, the implementations of many state-of-the-art ML algorithms are available in general programming languages (e.g., Python) and statistical programming languages (e.g., R) without any cost. Therefore, it is becoming easier for measurement specialists to learn and use ML toolkits without starting from scratch. Still, there are other potential barriers to embracing ML into the assessment and measurement community. This section debunks further some common myths and provides practical recommendations.

Misconception 1: ML Only Works for Big Data

A common misconception is that ML algorithms only work for big data (Cui, 2021). The tutorial shows that ML and statistics share many algorithms (e.g., regressions, principal component analysis, cluster analysis). Many of these algorithms can be adjusted to suit different data sizes. While prediction accuracy tends to increase as data size increases, ML models can still offer very good predictions with smaller data sets. For example, we used a data set of 750 records in the examples provided in this paper. More importantly, the data size requirement is more a function of the problem complexity and data quality than the ML algorithm. If a problem has a relatively simple underlying pattern, sample sizes in hundreds or thousands may be sufficient for ML to capture the underlying pattern. In contrast, if the underlying problem is complex (e.g., building a general language model) and the data contain lots of noise, big data (e.g., the entire Wikipedia) may be necessary to capture the underlying pattern. Additionally, unlike traditional statistics, it is acceptable for a ML model to have more parameters than data size (due to regularization). Sometimes, over-parameterized ML models can lead to better performance in the testing set (Belkin et al., 2019). For these reasons, in a systematic review, Balki et al.'s (2019) recommended using a post hoc method to determine the data size requirement: using increasingly higher proportions of data (20%, 30%, 40% etc.) to train the model and then plotting the errors for each data size. This data size versus error graph can be used to determine the data size needed for the desired error rate.

Misconception 2: Only Large Enterprises Can Afford ML

Some practitioners may be concerned that ML solutions are not affordable for smaller enterprises. The impression is that all ML problems are highly complex and require big data, a network of supercomputers and a devoted data science team. The reality is that ML can be applied to both simple and complex problems. For example, many problems in educational measurement (e.g., short-answer scoring, topic analysis, and risk prediction) can be solved efficiently with existing ML packages. Furthermore, in modern times, various well-developed ML packages and big data pretrained models are available for free. Practitioners do not need to “reinvent the wheel” to apply ML. Our demonstration is an example that shows how to apply a state-of-the-art NLP model to help short-answer marking. Accordingly, we recommend that smaller enterprises use similar approaches to explore ML solutions.

Misconception 3: One ML Algorithm Is Enough for Every Problem

Some ML beginners think every problem can be solved with a powerful ML algorithm. However, the effectiveness of a ML algorithm depends heavily on the data and the underlying problem. No matter how powerful a ML algorithm is, it will fall short if the data do not contain enough relevant information to the problem. For this reason, ECD is essential for the successful applications of ML in educational measurement. Additionally, a ML algorithm's effectiveness for a given problem depends on the problem's nature and complexity. Deep learning models outshine other models when the problem is complex and the data is big. When the problem is relatively simple or the data is not large enough, many simple models already perform well. The extra complexity of deep learning does not add much (sometimes, it is even worse due to being harder to train). However, when problem complexity and data size increase, many traditional models cannot keep up, but deep learning models scale very well due to the flexibility and scalability of their architecture. Therefore, we recommend that ML beginners start with the random forest and boosting algorithms for structured supervised problems and use deep learning for complex and unstructured NLP, video, and audio processing tasks. It is important to note that this recommendation is only a heuristic for beginners to get started with ML. No theory can predict exactly which algorithm will perform the best for a given problem.

Misconception 4: ML Solutions Are Developed to Replace Human Experts Completely

In the field of educational measurement, ML rarely completely replaces human experts. Instead, most ML solutions augment human experts by doing those tedious but less complicated tasks. The cost-effectiveness and reliability of ML in doing such tasks allow human experts to focus on tasks requiring higher levels of judgment. However, in practice, fully automated ML solutions are often risky and controversial. That is partly because ML still cannot reach human performance on specific tasks (e.g., language comprehension) and partly because when mistakes are made, society tends to be more critical of computers' mistakes than humans' mistakes. For this reason, we recommend using ML to augment human experts rather than replace them.

Misconception 5: ML Models Are Black Boxes

Many researchers feel uncomfortable about ML because of its “atheoretical” nature: theories are not always needed to specify the model, and it is hard to understand what knowledge is extracted from the data. While this may be true in early ML research, modern ML has a variety of techniques that shed light on how ML models arrive at their final decisions. For example, meaningful features (e.g., number of grammatical errors) can be extracted from raw data using pretrained models, and feature importance can be calculated to help researchers understand which features contribute the most to the final decision. In addition, many ML models are based on the decision tree algorithm, which makes the ML decision-making process explicit. Even complex deep neural networks have algorithms that help trace which parts of the input contribute the most to the final decision. In some situations, it may also be acceptable to prefer a less accurate but more interpretable model. In the examples provided in this tutorial, we

compared models built on word n -grams and character n -grams. The latter models performed better, but the former models are easier to interpret. If needed, alternative solutions, such as correcting for misspellings during the data preprocessing steps, could combine the power of both approaches.

The implication for measurement professionals is that ML models' validity can be studied with careful planning. Consequently, we recommend analyzing the final model's feature importance whenever possible.

Limitations and Significance

Due to space limitations, this tutorial does not cover *Reinforcement Learning*, a unique ML branch that deserves an independent introduction. To conclude, the digitalization of assessment results in an explosion of data volume, variety, and velocity, which demands cost-effective, reliable, and flexible solutions. ML is a very appealing tool for measurement professionals to achieve these goals. This tutorial aims to help measurement professionals with diverse backgrounds develop practical ML literacy: the ability to understand ML-related papers at a conceptual level, conduct basic ML studies with standard ML procedures, and make ML-related decisions in practice.

Notes

¹Reinforcement learning is another major class of ML that is not covered in this tutorial.

²<https://github.com/louisfb01/start-machine-learning>

³<https://medium.com/tag/data-science>

⁴<https://www.reddit.com/r/learnmachinelearning/>

⁵<https://stackoverflow.com/questions/tagged/machine-learning>

⁶We acknowledge that preprocessing/altering raw candidate responses may raise ethical concerns. Consequently, we only use automated marking to augment human markers rather than replace human markers. Ultimately, it is the human markers who make the final decision.

⁷Note that the test sets for supervised and unsupervised methods are slightly different due to the centroid responses in the unsupervised test set being removed. Nonetheless, the overall results still stand.

ORCID

Rui Nie  <https://orcid.org/0000-0001-6130-0507>

Qi Guo  <https://orcid.org/0000-0002-8685-6015>

Maxim Morin  <https://orcid.org/0000-0002-8683-1213>

References

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., ... Zheng, X. (2016, May 31). Tensorflow: A system for large-scale machine learning. Retrieved from <https://arxiv.org/abs/1605.08695>
- Almusharraf, N., & Alotaibi, H. (2020). Gender-based EFL writing error analysis using human and computer-aided approaches. *Educational Measurement: Issues and Practice*, 40(2), 60–71. Portico. <https://doi.org/10.1111/emip.12413>
- Anderson, D., Rowley, B., Stegenga, S., Irvin, S., & Rosenber, J. M. (2020). Evaluating content-related validity evidence using a test-based machine learning procedure. *Educational Measurement: Issues and Practice*, 39(4), 53–64. <https://doi.org/10.1111/emip.12314>
- Balki, I., Amirabadi, A., Levman, J., Martel, A. L., Emersic, Z., Meden, B., Garcia-Pedrero, A., Ramirez, S. C., Kong, D., Moody, A. R., & Tyrrell, P. N. (2019). Sample-size determination methodologies for machine learning in medical imaging research: A systematic review. *Canadian Association of Radiologists Journal*, 70(4), 344–353. <https://doi.org/10.1016/j.carj.2019.06.002>
- Belkin, M., Hsu, D., Ma, S., & Mandal, S. (2019). Reconciling modern machine-learning practice and the classical bias-variance trade-off. *Physical Sciences*, 116(32), 15849–15854. <https://doi.org/10.1073/pnas.1903070116>
- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, 281–305.
- Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24, 123–140. <https://doi.org/10.1007/BF00058655>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5–32. <https://doi.org/10.1023/A:1010933404324>
- Burkhardt, A., Lottridge, S., & Woolf, S. (2020). A rubric for the detection of students in crisis. *Educational Measurement: Issues and Practice*, 40(2), 72–80. Portico. <https://doi.org/10.1111/emip.12410>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794). New York, NY, USA: ACM. <https://doi.org/10.1145/2939672.2939785>
- Chollet, F. (2015). Keras. GitHub. Retrieved from <https://github.com/fchollet/keras>
- Clark, K., & Manning, C. D. (2016). Improving coreference resolution by learning entity-level distributed representations. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. <https://doi.org/10.18653/v1/p16-1061>
- Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37–46.
- Cohen, J. (1968). Weighted kappa: Nominal scale agreement with provision for scaled disagreement or partial credit. *Psychological Bulletin*, 70, 213–220.
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.
- Cui, Z. (2021). Machine learning and small data. *Educational Measurement: Issues and Practice*, 40(4), 8–12. <https://doi.org/10.1111/emip.12472>
- Ercikan, K., & McCaffrey, D. F. (2022). Optimizing implementation of artificial-intelligence-based automated scoring: An evidence centered design approach for designing assessments for AI-based scoring. *Journal of Educational Measurement*, 59(3), 272–287. Portico. <https://doi.org/10.1111/jedm.12332>
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.
- Ferrara, S. (2017). A framework for policies and practices to improve test security programs: Prevention, detection, investigation, and resolution (PDIR). *Educational Measurement: Issues and Practice*, 36(3), 5–23. <https://doi.org/10.1111/emip.12151>
- Frank, E., Hall, M. A., & Witten, I. H. (2016). *The WEKA Workbench. Online Appendix for "Data Mining: Practical Machine Learning Tools and Techniques"* (4th edn.). Burlington: Morgan Kaufmann.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29, 1189–1232.
- Hao, J. (2021). Supervised Machine Learning. In A. A. von Davier, R. J. Mislevy, & J. Hao (Eds.), *Computational psychometrics: New methodologies for a new generation of digital learning and assessment with examples in R and Python* (pp. 159–171). Springer. <https://doi.org/10.1007/978-3-030-74394-9>
- Hao, J., & Mislevy, R. J. (2021). A data science perspective on computational psychometrics. *Computational Psychometrics: New Methodologies for a New Generation of Digital Learning and Assessment*, 133–158. https://doi.org/10.1007/978-3-030-74394-9_8
- Honnibal, M., & Montani, I. (2017). spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing. Sentometrics Research.
- Huang, Y., & Khan, S. M. (2021). Advances in AI and machine learning for education research. In A. A. von Davier, R. J. Mislevy, & J. Hao (Eds.), *Computational psychometrics: New methodologies for a new generation of digital learning and assessment with examples in R and Python* (pp. 195–208). Springer. <https://doi.org/10.1007/978-3-030-74394-9>

- McNamara, D. S., Louwerse, M. M., McCarthy, P. M., & Graesser, A. C. (2010). Coh-Metrix: Capturing linguistic features of cohesion. *Discourse Processes*, 47(4), 292–330.
- Mislevy, R. J., Steinberg, L. S., & Almond, R. G. (2003). On the structure of educational assessments. *Measurement: Interdisciplinary Research and Perspectives*, 1, 3–62. https://doi.org/10.1207/S15366359MEA0101_02
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., & Passos, A. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Pennington, J., Socher, R., & Manning, C. D. (2014). Glove: Global vectors for word representation. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, Qatar, pp. 1532–1543. <https://doi.org/10.3115/v1/D14-1162>
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2017). CatBoost: unbiased boosting with categorical features (Version 5). arXiv. <https://doi.org/10.48550/ARXIV.1706.09516>
- Rafatbakhsh, E., Ahmadi, A., Moloodi, A., & Mehrpour, S. (2021). Development and validation of an automatic item generation system for English idioms. *Educational Measurement: Issues and Practice*, 40(2), 49–59.
- Russell, S., & Norvig, P. (2010). *Artificial intelligence: A modern approach*. (3rd edn.). Upper Saddle River: Prentice-Hall.
- San Pedro, M. O. Z., & Baker, R. S. (2021). Knowledge inference models used in adaptive learning. In A. A. von Davier, R. J. Mislevy, & J. Hao (Eds.), *Computational psychometrics: new methodologies for a new generation of digital learning and assessment with examples in R and Python* (pp. 61–77). Springer. <https://doi.org/10.1007/978-3-030-74394-9>
- Savi, A. O., Cornelisz, I., Sjerps, M. J., Greup, S. L., Bres, C. M., & van Klaveren, C. (2021). Balancing trade-offs in the detection of primary schools at risk. *Educational Measurement: Issues and Practice*, 40(3), 110–124. Portico. <https://doi.org/10.1111/emip.12433>
- Shane, L., & Marcus, H. (2007, Jun 15). A collection of definitions of intelligence. Retrieved from: <https://arxiv.org/abs/0706.3639>
- Sheehan, K. M. (2017). Validating automated measures of text complexity. *Educational Measurement: Issues and Practice*, 36(4), 35–43. <https://doi.org/10.1111/emip.12155>
- von Davier, A. A., Mislevy, R. J., & Hao, J. (Eds.). (2021). *Computational psychometrics: New methodologies for a new generation of digital learning and assessment with examples in R and Python*. Springer. <https://doi.org/10.1007/978-3-030-74394-9>
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T. L., Gugger, S., ... Rush, A. M. (2020, Jul 14). HuggingFace's transformers: State-of-the-art natural language processing. Retrieved from <https://arxiv.org/abs/1910.03771>
- Wong, P. C. (2021). Unsupervised machine learning. In A. A. von Davier, R. J. Mislevy, & J. Hao (Eds.), *Computational psychometrics: New methodologies for a new generation of digital learning and assessment with examples in R and Python* (pp. 173–193). Springer. <https://doi.org/10.1007/978-3-030-74394-9>
- Yang, Z., Dai, Z., Yang, Y., Carbonell, J., Salakhutdinov, R., & Le, Q. V. (2020, Jan 1). XLNet: Generalized autoregressive pretraining for language understanding. Retrieved from <https://arxiv.org/abs/1906.08237>
- Zhou, Z. -H. (2012). *Ensemble methods: Foundations and algorithms* (1st ed.). Chapman and Hall/CRC. <https://doi.org/10.1201/b12207>

Appendix A.

Appendix A contains lists of machine learning glossaries covered in the main document. The aim is to help readers quickly look up a machine learning term A1-A4.

Table A1
General ML Glossaries

Term	Definition
Machine learning	“A field of study that gives computers the ability to learn without being explicitly programmed” (Cui, 2021).
Supervised learning	In supervised learning, the machine is given example input (feature)—output (label) pairs, and the machine learns a function that maps from input (feature) to output (label) (Russell & Norvig, 2010).
Unsupervised learning	In unsupervised learning, the machine learns patterns in the input (feature) even though no explicit feedback (label) is supplied (Russell & Norvig, 2010).
Classification	In supervised learning, if the output (label) is a categorical variable, it is called a classification problem.
Regression	In supervised learning, if the output (label) is a continuous variable, it is called a regression problem.
Pipeline	A machine learning pipeline is a construct that chains a sequence of data processing steps to form an end-to-end solution to a problem.

(Continued)

Table A1
Continued

Term	Definition
Transfer learning	Transfer learning, simply put, is to train a machine learning model on one task (usually involving a big database), and then apply the pretrained model on a different but related task. This way, the knowledge learned from one task is stored in the model and transferred to another similar task. The pretrained model can be a starting point in the new task, meaning that the parameters in the pretrained model will be used only as initialization values and later be trained by the data in the new task. Alternatively, the pretrained model can be “frozen,” meaning that all parameters in the pretrained models will be kept as they are and will not be altered in the new task. In other words, the model is applied to the new task as is, and there will not be any new training process.
Hyperparameters	In machine learning, hyperparameters are a set of parameters that control the learning process, such as the learning rate and model complexity. They need to be specified before training, unlike other parameters that can be optimized during the training phase. The existence of hyperparameters is one of the reasons why grid search and random search are needed; because hyperparameters cannot be automatically learned through training, we need to prespecify a set of values of hyperparameters to try out and search for the best values.
Search space	A search space is a set or domain through which an algorithm searches, usually to find the best solution. In machine learning, the search space usually consists of different combinations of models and hyperparameters, the computer will go through the search space and find the best-performing combination for a machine learning problem.
Grid search	A grid search is to exhaustively search through all combinations specified in the search space. It is often used when the search space is a small finite set, so the time and computational resources spent on the exhaustive search are affordable.
Random search	When the search space is a large finite set or an infinite space, grid search becomes too costly in computational time and resources. In these cases, random search is often used. As the name suggests, a random search does not do an exhaustive search of the whole search space, but instead draws a random subset from it. The researcher predefines the size and distribution of the random draws.
Training set	The training set is a data set used to train the model parameters.
Validation set	The validation set is a separate data set from the training set; it is used for evaluating the model trained with the training set. The purpose of a validation set is to prevent the model from overfitting to the training set and ensure that it generalizes well to other data sets from the same distribution. During a grid search/random search, different model and hyperparameter combinations will be evaluated using the validation set, and the best-performing combination is selected based on the evaluation result. Thus, the validation set is also used for model selection and hyperparameter tuning. Typically, when a model’s performance is much worse in the validation set than in the training set, it indicates that the model is overfitting to the training set.
Test set	Since the training set is used for training model parameters, and the validation set is used for model selection and hyperparameter tuning, another independent data set (but with the same distribution) is needed to evaluate the performance of the final chosen model, and thus the existence of the test set.
Hold-out validation	Hold-out validation means that a subset of the data is held out for validation. In other words, the data set is split into two parts: the training set and the validation set.

(Continued)

Table A1*Continued*

Term	Definition
K-fold cross-validation	In a k -fold cross-validation, the data set is separated into k subsets, and the model training and evaluation will be done k times; each time, a different subset is used as the validation set, and the other $k - 1$ subsets form the training set. In the end, the k evaluation results will be aggregated to form a more stable and reliable evaluation result. The “ k ” here is a number that will be predefined by the researcher. Usually, $k = 5$ is used. When $k = 1$, it is a hold-out validation.
Underfitting	Underfitting means that the model cannot accurately predict the label. If a model has a bad performance on the training set, it usually indicates that the model is underfitting.
Overfitting	Overfitting means that the model fits too closely to the training set, but fails to generalize well to other data sets from the same distribution. If a model's performance is much worse in the validation set than that in the training set, it usually indicates that the model is overfitting to the training set.
Regularization	Regularization is a technique used to reduce overfitting. It usually works by putting some additional constraints or penalties on the parameters in the model to prevent the model from getting too complicated and not generalizable.

Table A2*Feature Extraction Glossaries*

Term	Definition
N-gram	N -gram means a continuous sequence of n units, the unit can be character, word, etc. When $n = 1$, it is called unigram; when $n = 2$, it is called bigram; when $n = 3$, it is called trigram.
Bag of words	A method to represent a piece of text by a vector of token frequencies, the token can be words, characters, n -grams, etc.
TFIDF	TFIDF is short for <i>Term Frequency-Inverse Document Frequency</i> , it is a weighting method to apply weights to different tokens (terms) according to their frequencies in the whole text data.
Text encoding Word embedding	In NLP, text encoding usually means mapping a text to a numeric vector. The numeric vector a word can be mapped to/represented by.

Table A3*ML Model Glossaries*

Term	Definition
Logistic regression	Logistic regression is a type of regression model usually used to model the probability of an event. The model's input is the same as linear regression, but the output has a range between 0 to 1.
Support vector machine	A type of supervised learning algorithm. The general idea of the algorithm is to first nonlinearly map the input data into a high-dimensional feature space, then try to find an optimal hyperplane that will separate data that belong to different classes as much as possible (Cortes & Vapnik, 1995).
Ensemble methods	“Ensemble methods train multiple learners to solve the same problem. In contrast to ordinary learning approaches, which try to construct one learner from training data, ensemble methods try to construct a set of learners and combine them” (Zhou, 2012). The ensemble methods are designed to improve prediction accuracy and stability. Two common types of ensemble methods are bagging and boosting.

(Continued)

Table A3*Continued*

Term	Definition
Bagging algorithms	Bagging is the acronym for “bootstrap aggregating.” “Bagging predictors is a method for generating multiple versions of a predictor and using these to get an aggregated predictor. The aggregation averages over the versions when predicting a numerical outcome and does a plurality vote when predicting a class” (Breiman, 1996).
Boosting algorithms	Boosting also works by training a series of predictors and aggregating them in the end, but one major difference from bagging is that boosting trains the predictors sequentially and adaptively: the later predictors receive feedback from earlier predictors and make adjustments accordingly, the later predictors will focus more on data that are misclassified by earlier predictors. The aggregation phase is also more complicated than that in bagging algorithms. Usually, the final aggregated predictor is a weighted combination of the individual predictors; the weights are usually determined by their performance (Zhou, 2012).
Random forest	Random forest is a typical bagging algorithm. In the training phase, bootstrapping is used to generate a collection of data samples, and a decision tree is fit to each of those data samples. In the aggregation phase, each decision tree casts a vote on the prediction, and the most popular vote will be selected as the prediction of the random forest predictor (Breiman, 2001).
Gradient boosting	Gradient boosting is a subclass of the boosting algorithm. It trains a series of decision trees sequentially and aggregates the results. It is called gradient boosting because the algorithm mainly uses the gradient descent technique in the optimization process (Friedman, 2001).
XGBoost	XGBoost stands for “Extreme Gradient Boosting.” It is an open-source software library that implements optimized distributed gradient boosting algorithms. It mainly implements the original gradient boosting algorithm but also introduces some slight enhancements. The ‘extreme’ here refers to an engineering goal to push the limits of computation resources. The library is popular due to its efficiency, flexibility, scalability, and portability (Chen & Guestrin, 2016).
CatBoost	CatBoost is another high-performance open-source software library that implements gradient boosting algorithms, but it introduces some new enhancements, especially in handling categorical features. The “Cat” in “CatBoost” refers to its categorical features support (Prokhorenkova et al., 2017).
Neural networks	Neural networks here refer to <i>Artificial Neural Networks</i> ; they are machine learning models inspired by biological neural networks. The model comprises of a sequence of layers, and each layer consists of a set of nodes (artificial neurons). The nodes in two neighboring layers are interconnected with each other. Thus the whole model is a network of artificial neurons. The first layer is called input layer, it receives input data. The last layer is called output layer, which outputs prediction result. The layers between the input and output layers are called hidden layers. Each node has its own (trainable) weights, and also carries a function called the activation function. When data pass through one layer of nodes, they go through these activation functions and are aggregated with their corresponding weights in the nodes in the next layer. The complexity and flexibility of its structure enable it to model complex relationships. Therefore, in supervised learning problems, when the training data is big enough, neural network models are often the best-performing models. Logistic regression can be a special case of a neural network model with no hidden layers.
Deep learning	Deep learning, or a deep neural network, is a neural network model with more than three layers, input and output layers included.
Convolutional neural networks	Convolutional neural networks are a set of neural network models that have one or more convolutional layers. The convolutional layer has a unique structure that is inspired by the organization of the animal visual cortex.
XLNet	XLNet is currently one of the state-of-the-art deep learning models in text classification field.
K-means clustering	K-means clustering is an unsupervised learning algorithm. It aims to group data records (rows) into a predefined number of clusters. The mean (also called centroid) of all data points in each cluster is given to represent the whole cluster.

Table A4**ML Evaluation Glossaries**

Term	Definition
Accuracy	The percentage of correct predictions.
Kappa	Also known as Cohen's Kappa, a statistic to measure interrater reliability for binary label variable.
F-score	The harmonic mean of precision and recall.
ROC curve and AUC	ROC stands for <i>Receiver Operating Characteristic</i> ; the ROC curve is a plot of true positive rate against false positive rate at different classification thresholds. AUC means <i>Area Under the Curve</i> . It measures the total area under the ROC curve, and evaluates a classifier's overall ability to distinguish between classes.
Confusion matrix	A table with two rows and two columns reporting the values of true positives, false negatives, false positives, and true negatives.